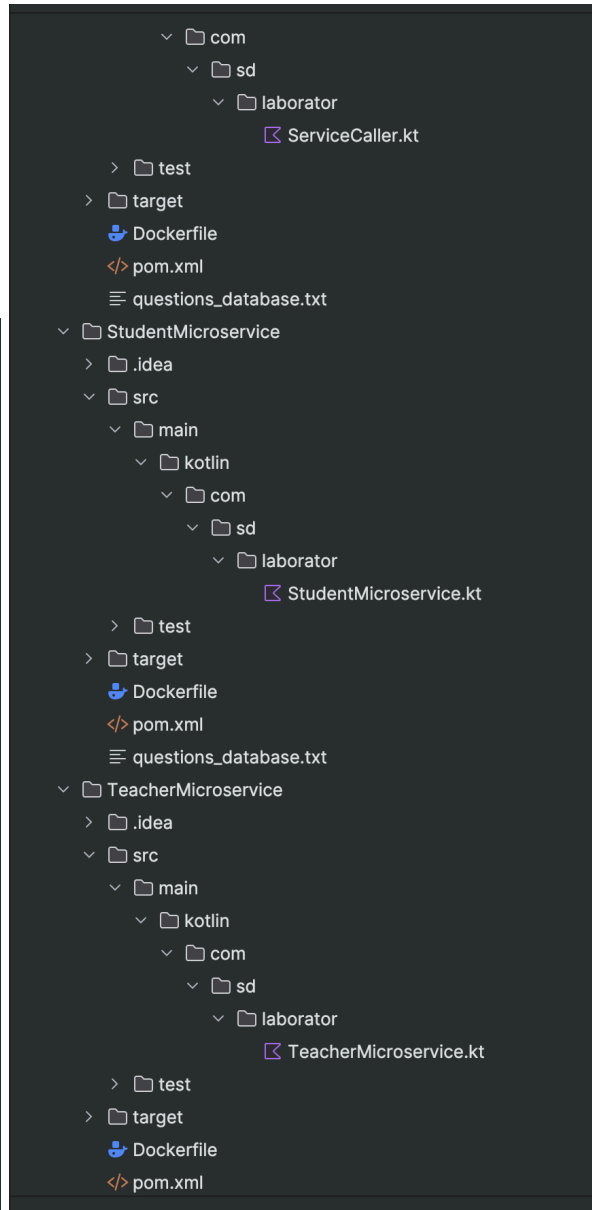
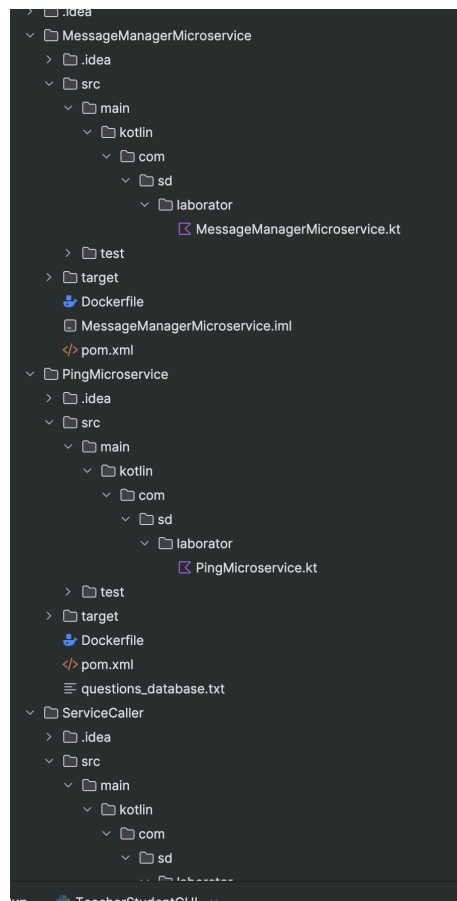


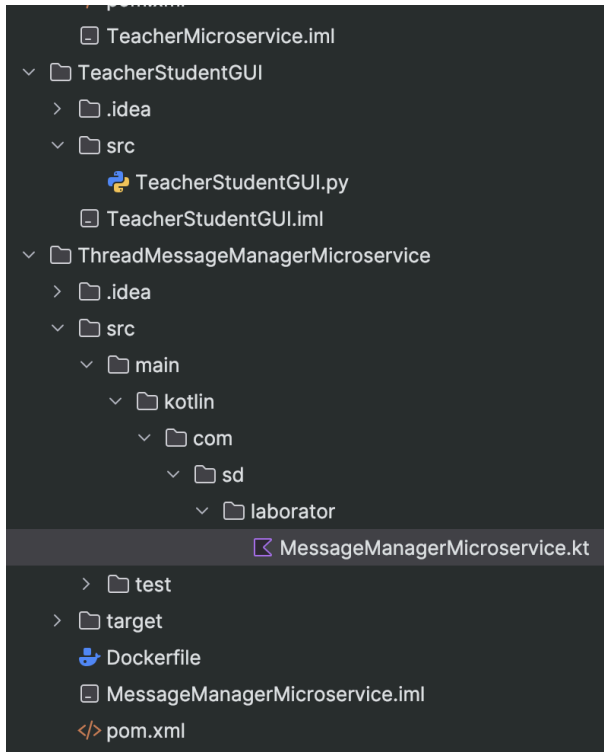
Pornind de la aplicatia exemplu de chat din laboratorul 8, să se modifice aceasta prin introducerea unui procesor de flux care implementează un mecanism de "bătăie de inimă" ce trimite mesaje false, in scopul verificării functionării corecte ale celorlalte microservicii. Dacă un serviciu nu functioneaza, se va trimite o cerere de replicare a acestuia unui procesor de flux dedicat.

Dacă se consideră necesar se mai pot crea oricâte servicii se doreste conform gândirii studentului. Se vor respecta principiile SOLID pentru microservicii. La prezentare se va arăta diagrama de clase si se va explica aceasta împreună cu gradul de respectare a principiilor. Apoi va urma discutia pe cod.

Notite

1. Vezi in laboratorul 8 configurarea cu tkinter





MessageManagerMicroservice

```
package com.sd.laborator

import java.io.BufferedReader
import java.io.InputStreamReader
import java.net.InetAddress
import java.net.ServerSocket
import java.net.Socket
import kotlin.concurrent.thread

class MessageManagerMicroservice {
    private val subscribers: HashMap<Int, Socket>
    private lateinit var messageManagerSocket: ServerSocket

    companion object Constants {
        const val MESSAGE_MANAGER_PORT = 1500
    }

    init {
        subscribers = hashMapOf()
    }

    private fun broadcastMessage(message: String, except: Int) {
        subscribers.forEach {
            it.takeIf { it.key != except }
        }
    }
}
```

```

        ?.value?.getOutputStream()?.write((message +
"\n").toByteArray())
    }
}

private fun respondTo(destination: Int, message: String) {
    subscribers[destination]?.getOutputStream()?.write((message +
"\n").toByteArray())
}

public fun run() {
    // se porneste un socket server TCP pe portul 1500 care asculta pentru
conexiuni
    messageManagerSocket = ServerSocket(MESSAGE_MANAGER_PORT)
    println("MessageManagerMicroservice se executa pe portul:
${messageManagerSocket.localPort}")
    println("Se asteapta conexiuni si mesaje...")

    while (true) {
        // se asteapta conexiuni din partea clientilor subscriberi
        val clientConnection = messageManagerSocket.accept()

        // se porneste un thread separat pentru tratarea conexiunii cu
clientul
        thread {
            println("Subscriber conectat:
${clientConnection.inetAddress.hostAddress}:${clientConnection.port}")

            // adaugarea in lista de subscriberi trebuie sa fie atomica!
            synchronized(subscribers) {
                subscribers[clientConnection.port] = clientConnection
            }

            val bufferedReader =
BufferedReader(InputStreamReader(clientConnection.inputStream))

            while (true) {
                // se citeste raspunsul de pe socketul TCP
                val receivedMessage = bufferedReader.readLine()

                // daca se primeste un mesaj gol (NULL), atunci inseamna ca
cealalta parte a socket-ului a fost inchisa
                if (receivedMessage == null) {
                    // deci subscriber-ul respectiv a fost deconectat
                    println("Subscriber-ul ${clientConnection.port} a fost
deconectat.")

                    synchronized(subscribers) {
                        subscribers.remove(clientConnection.port)
                    }
                }
            }
        }
    }
}

```

```

        bufferedReader.close()
        clientConnection.close()
        break
    }

    println("Primit mesaj: $receivedMessage")

    // VERIFICARE CU HEARTBEAT SERVICE
    if(receivedMessage.startsWith("ping")) {
clientConnection.getOutputStream().write("pong\n".toByteArray())
        continue
    }

    val (messageType, messageDestination, messageBody) =
receivedMessage.split(" ", limit = 3)

    when (messageType) {
        "intrebare" -> {
            // tipul mesajului de tip intrebare este de forma:
            // intrebare <DESTINATIE_RASPUNS>
<CONTINUT_INTREBARE>
            broadcastMessage("intrebare ${clientConnection.port}
$messageBody", except = clientConnection.port)
        }
        "raspuns" -> {
            // tipul mesajului de tip raspuns este de forma:
            // raspuns <CONTINUT_RASPUNS>
            respondTo(messageDestination.toInt(), messageBody)
        }
    }
}
}
}
}

fun main(args: Array<String>) {
    val messageManagerMicroservice = MessageManagerMicroservice()
    messageManagerMicroservice.run()
}

```

PingMicroservice

```

package com.sd.laborator

import java.io.BufferedReader

```

```

import java.io.InputStreamReader
import java.net.*
import kotlin.concurrent.thread
import kotlin.system.exitProcess

class PingMicroservice {
    private lateinit var messageManagerSocket: Socket
    private lateinit var studentMicroserviceSocket: Socket
    private lateinit var serviceCallerSocket: Socket

    companion object Constants {
        // pentru testare, se foloseste localhost. pentru deploy, server-ul
        // socket (microserviciul MessageManager) se identifica dupa un "hostname"
        // acest hostname poate fi trimis (optional) ca variabila de mediu
        val MESSAGE_MANAGER_HOST = System.getenv("MESSAGE_MANAGER_HOST") ?:
"localhost"
        const val MESSAGE_MANAGER_PORT = 1500
    }

    private fun connectToServiceCaller() {
        try {
            serviceCallerSocket = Socket(
                System.getenv("SERVICE_CALLER_HOST") ?: "localhost",
                1660
            )
            serviceCallerSocket.soTimeout = 3000
            println("M-am conectat la serviceCallerSocket!")
        } catch (e: Exception) {
            println("Nu ma pot conecta la serviceCallerSocket!")
            exitProcess(1)
        }
    }

    private fun connectToMessageManager() {
        try {
            messageManagerSocket = Socket(
                MESSAGE_MANAGER_HOST,
                MESSAGE_MANAGER_PORT
            )
            messageManagerSocket.soTimeout = 3000
            println("M-am conectat la MessageManager!")
        } catch (e: Exception) {
            println("Nu ma pot conecta la MessageManager!")
            exitProcess(1)
        }
    }

    private fun connectToStudentMicroservice() {
        try {

```

```

        studentMicroserviceSocket = Socket(
            System.getenv("STUDENT_MICROSERVICE_HOST") ?: "localhost",
            1650
        )
        studentMicroserviceSocket.soTimeout = 3000
        println("M-am conectat la StudentMicroservice!")
    } catch (e: Exception) {
        println("Nu ma pot conecta la StudentMicroservice!")
        exitProcess(1)
    }
}

fun run() {
    connectToServiceCaller()

    // microserviciul se inscrie in lista de "subscribers" de la
    MessageManager prin conectarea la acesta
    connectToMessageManager()
    // ne conectam pur si simplu la student microservice
    connectToStudentMicroservice()

    while (true) {
        // se asteapta raspuns de la MessageManager
        val messageManagerBufferReader =
            BufferedReader(InputStreamReader(messageManagerSocket.inputStream))
        val studentMicroserviceBufferReader =
            BufferedReader(InputStreamReader(studentMicroserviceSocket.inputStream))

        // CATE UN TRY-CATCH PENTRU FIECARE SERVICIU MONITORIZAT PRIN
        HEARTBEAT

        // MESSAGE MANAGER
        try {
            messageManagerSocket.getOutputStream().write("ping\n".toByteArray())
            val receivedResponse = messageManagerBufferReader.readLine()

            if(receivedResponse != null)
                print("Raspuns messageManager >> " + receivedResponse)

            // se trimite raspunsul inapoi clientului apelant
            if (receivedResponse.startsWith("pong")) {
                println("MessageManager functional")
            } else {
                println("MessageManager NU functioneaza!")
                // aici se va apela alt serviciu care va rula comanda "java
                -jar [nume_pachet].jar" trimitandu-i numele serviciului pe care sa il ruleze
                // Runtime.getRuntime().exec("ls")
            }
        } catch (e: Exception) {
            println("Exception in MessageManager")
        }
    }
}

```

```

serviceCallerSocket.getOutputStream().write("message_manager".toByteArray())
    }
    } catch (e: SocketTimeoutException) {
        println("Eroare la MessageManager")

//serviceCallerSocket.getOutputStream().write("message_manager".toByteArray())
    }

    // STUDENT MICROSERVICE
    try {

studentMicroserviceSocket.getOutputStream().write("ping\n".toByteArray())
        Thread.sleep(1000)
        val receivedResponse =
studentMicroserviceBufferedReader.readLine()

        if(receivedResponse != null)
            print("Raspuns studentMicroservice >> " + receivedResponse)

        // se trimite raspunsul inapoi clientului apelant
        if (receivedResponse.startsWith("pong")) {
            println("StudentMicroservice functional")
        } else {
            println("StudentMicroservice NU functioneaza!")
            // aici se va apela alt serviciu care va rula comanda "java
-jar [nume_pachet].jar" trimitandu-i numele serviciului pe care sa il ruleze
            // Runtime.getRuntime().exec("ls")

serviceCallerSocket.getOutputStream().write("student_microservice".toByteArray(
))
        }
        } catch (e: SocketTimeoutException) {
            println("Eroare la StudentMicroservice")

//serviceCallerSocket.getOutputStream().write("student_microservice".toByteArra
y())
        }

        // ping every second
        Thread.sleep(1000)
    }
}

fun main(args: Array<String>) {
    val pingMicroservice = PingMicroservice()
    pingMicroservice.run()
}

```



```
}
```

ServiceCaller

```
package com.sd.laborator

import java.io.BufferedReader
import java.io.InputStreamReader
import java.net.ServerSocket

class ServiceCaller {
    fun run() {
        val serviceCallerSocket = ServerSocket(1660) //pentru heartbeat
        println("Se asteapta conectarea serviciului de heartbeat")

        println(" se executa pe portul: ${serviceCallerSocket.localPort}")
        println("Se asteapta mesaje...")

        while (true) {
            val heartbeatConnection = serviceCallerSocket.accept()
            val heartbeatReader =
                BufferedReader(InputStreamReader(heartbeatConnection.inputStream))
            val message = heartbeatReader.readLine()
            println("Primit mesaj: $message")
            if (message.startsWith("message_manager")) {
                Runtime.getRuntime().exec("java -jar message_manager.jar")
            }
            else if (message.startsWith("student_microservice")) {
                Runtime.getRuntime().exec("java -jar student_microservice.jar")
            }
            else if (message == null) {
                heartbeatReader.close()
                serviceCallerSocket.close()
                break
            }
        }
    }
}

fun main(args: Array<String>) {
    val serviceCaller = ServiceCaller()
    serviceCaller.run()
}
```

StudentMicroservice

```
package com.sd.laborator
```

```

import java.io.BufferedReader
import java.io.File
import java.io.InputStreamReader
import java.lang.Exception
import java.net.ServerSocket
import java.net.Socket
import kotlin.concurrent.thread
import kotlin.system.exitProcess

class StudentMicroservice {
    // intrebarile si raspunsurile sunt mentinute intr-o lista de perechi de
    // forma:
    // [<INTREBARE 1, RASPUNS 1>, <INTREBARE 2, RASPUNS 2>, ... ]
    private lateinit var questionDatabase: MutableList<Pair<String, String>>
    private lateinit var messageManagerSocket: Socket
    private lateinit var studentMicroserviceSocket: ServerSocket

    init {
        val databaseLines: List<String> =
            File("questions_database.txt").readLines()
        questionDatabase = mutableListOf()

        /*
        "baza de date" cu intrebari si raspunsuri este de forma:

        <INTREBARE_1>\n
        <RASPUNS_INTREBARE_1>\n
        <INTREBARE_2>\n
        <RASPUNS_INTREBARE_2>\n
        ...
        */
        for (i in 0..(databaseLines.size - 1) step 2) {
            questionDatabase.add(Pair(databaseLines[i], databaseLines[i + 1]))
        }
    }

    companion object Constants {
        // pentru testare, se foloseste localhost. pentru deploy, server-ul
        // socket (microserviciul MessageManager) se identifica dupa un "hostname"
        // acest hostname poate fi trimis (optional) ca variabila de mediu
        val MESSAGE_MANAGER_HOST = System.getenv("MESSAGE_MANAGER_HOST") ?:
            "localhost"
        const val MESSAGE_MANAGER_PORT = 1500
    }

    private fun subscribeToMessageManager() {
        try {
            messageManagerSocket = Socket(MESSAGE_MANAGER_HOST,
                MESSAGE_MANAGER_PORT)

```

```

        println("M-am conectat la MessageManager!")
    } catch (e: Exception) {
        println("Nu ma pot conecta la MessageManager!")
        exitProcess(1)
    }
}

private fun respondToQuestion(question: String): String? {
    questionDatabase.forEach {
        // daca se gaseste raspunsul la intrebare, acesta este returnat
        apelantului
        if (it.first == question) {
            return it.second
        }
    }
    return null
}

public fun run() {
    studentMicroserviceSocket = ServerSocket(1650) //pentru heartbeat

    // microserviciul se inscrie in lista de "subscribers" de la
    MessageManager prin conectarea la acesta
    subscribeToMessageManager()

    println("StudentMicroservice se executa pe portul:
    ${messageManagerSocket.localPort}")
    println("Se asteapta mesaje...")

    val bufferedReader =
    BufferedReader(InputStreamReader(messageManagerSocket.inputStream))

    while (true) {
        val heartbeatConnection = studentMicroserviceSocket.accept()
        val heartbeatReader =
        BufferedReader(InputStreamReader(heartbeatConnection.inputStream))

        val message = heartbeatReader.readLine()
        println("Primit mesaj: $message")
        if (message.startsWith("ping")) {
            heartbeatConnection.getOutputStream().write("pong\n".toByteArray())
        }
        else if (message == null) {
            heartbeatReader.close()
            studentMicroserviceSocket.close()
            break
        }
    }
}

```

```

        // se asteapta intrebari trimise prin intermediarul "MessageManager"
        val response = bufferedReader.readLine()

        if (response == null) {
            // daca se primește un mesaj gol (NULL), atunci înseamnă ca
            // cealalta parte a socket-ului a fost închisa
            println("Microserviciul MessageService
            (${messageManagerSocket.port}) a fost oprit.")
            bufferedReader.close()
            messageManagerSocket.close()
            break
        }

        // se folosește un thread separat pentru tratarea întrebării primite
        thread {
            val (messageType, messageDestination, messageBody) =
            response.split(" ", limit = 3)

            when(messageType) {
                // tipul mesajului cunoscut de acest microserviciu este de
                forma:
                // întrebare <DESTINATIE_RASPUNS> <CONTINUT_INTREBARE>
                "intrebare" -> {
                    println("Am primit o întrebare de la
                    $messageDestination: \"${messageBody}\"")
                    var responseToQuestion = respondToQuestion(messageBody)
                    responseToQuestion?.let {
                        responseToQuestion = "raspuns $messageDestination
                        $it"
                    }
                    println("Trimit raspunsul: \"${response}\"")

                    messageManagerSocket.getOutputStream().write((responseToQuestion +
                    "\n").toByteArray())
                }
            }
        }
    }
}

fun main(args: Array<String>) {
    val studentMicroservice = StudentMicroservice()
    studentMicroservice.run()
}

```

TeacherMicroservice

```

package com.sd.laborator

import java.io.BufferedReader
import java.io.InputStreamReader
import java.net.*
import kotlin.concurrent.thread
import kotlin.system.exitProcess

class TeacherMicroservice {
    private lateinit var messageManagerSocket: Socket
    private lateinit var teacherMicroserviceServerSocket: ServerSocket

    companion object Constants {
        // pentru testare, se foloseste localhost. pentru deploy, server-ul
        // socket (microserviciul MessageManager) se identifica dupa un "hostname"
        // acest hostname poate fi trimis (optional) ca variabila de mediu
        val MESSAGE_MANAGER_HOST = System.getenv("MESSAGE_MANAGER_HOST") ?:
"localhost"
        const val MESSAGE_MANAGER_PORT = 1500
        const val TEACHER_PORT = 1600
    }

    private fun subscribeToMessageManager() {
        try {
            messageManagerSocket = Socket(MESSAGE_MANAGER_HOST,
MESSAGE_MANAGER_PORT)
            messageManagerSocket.soTimeout = 3000
            println("M-am conectat la MessageManager!")
        } catch (e: Exception) {
            println("Nu ma pot conecta la MessageManager!")
            exitProcess(1)
        }
    }

    public fun run() {
        // microserviciul se inscrie in lista de "subscribers" de la
        // MessageManager prin conectarea la acesta
        subscribeToMessageManager()

        // se porneste un socket server TCP pe portul 1600 care asculta pentru
        // conexiuni
        teacherMicroserviceServerSocket = ServerSocket(TEACHER_PORT)

        println("TeacherMicroservice se executa pe portul:
${teacherMicroserviceServerSocket.localPort}")
        println("Se asteapta cereri (intrebari)...")

        while (true) {

```

```

        // se asteapta conexiuni din partea clientilor ce doresc sa puna o
intrebare
        // (in acest caz, din partea aplicatiei client GUI)
        val clientConnection = teacherMicroserviceServerSocket.accept()

        // se foloseste un thread separat pentru tratarea fiecărei conexiuni
client
        thread {
            println("S-a primit o cerere de la:
${clientConnection.inetAddress.hostAddress}:${clientConnection.port}")

            // se citeste intrebarea dorita
            val clientBufferedReader =
BufferedReader(InputStreamReader(clientConnection.inputStream))
            val receivedQuestion = clientBufferedReader.readLine()

            // intrebarea este redirectionata catre microserviciul
MessageManager
            println("Trimit catre MessageManager: ${"intrebare
${messageManagerSocket.localPort} $receivedQuestion\n"}")
            messageManagerSocket.getOutputStream().write(("intrebare
${messageManagerSocket.localPort} $receivedQuestion\n").toByteArray())

            // se asteapta raspuns de la MessageManager
            val messageManagerBufferedReader =
BufferedReader(InputStreamReader(messageManagerSocket.inputStream))
            try {
                val receivedResponse = messageManagerBufferedReader.readLine()

                // se trimite raspunsul inapoi clientului apelant
                println("Am primit raspunsul: \"$receivedResponse\"")
                clientConnection.getOutputStream().write((receivedResponse +
"\n").toByteArray())
            } catch (e: SocketTimeoutException) {
                println("Nu a venit niciun raspuns in timp util.")
                clientConnection.getOutputStream().write("Nu a raspuns
nimeni la intrebare\n".toByteArray())
            } finally {
                // se inchide conexiunea cu clientul
                clientConnection.close()
            }
        }
    }
}

fun main(args: Array<String>) {
    val teacherMicroservice = TeacherMicroservice()
    teacherMicroservice.run()
}

```

```
}
```

[TeacherStudentGUI.py](#)

```
from tkinter import *
from tkinter import ttk
import threading
import socket

HOST = "localhost"
TEACHER_PORT = 1600

def resolve_question(question_text):
    # creare socket TCP
    sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

    # incercare de conectare catre microserviciul Teacher
    try:
        sock.connect((HOST, TEACHER_PORT))

        # transmitere intrebare - se deleaga intrebarea catre microserviciu
        sock.send(bytes(question_text + "\n", "utf-8"))

        # primire raspuns -> microserviciul Teacher foloseste coregrafia de
        # microservicii pentru a trimite raspunsul inapoi
        response_text = str(sock.recv(1024), "utf-8")

    except ConnectionError:
        # in cazul unei erori de conexiune, se afiseaza un mesaj
        response_text = "Eroare de conectare la microserviciul Teacher!"

    # se adauga raspunsul primit in caseta text din interfata grafica
    response_widget.insert(END, response_text)

def ask_question():
    # preluare text intrebare de pe interfata grafica
    question_text = question.get()

    # pornire thread separat pentru tratarea intrebarii respective
    # astfel, nu se blocheaza interfata grafica!
    threading.Thread(target=resolve_question, args=(question_text,)).start()

if __name__ == '__main__':
    # elementul radacina al interfetei grafice
    root = Tk()
```

```

root.title("Interactiune profesor-studenti")

# la redimensionarea ferestrei, cadrele se extind pentru a prelua spatiul ramas
root.columnconfigure(0, weight=1)
root.rowconfigure(0, weight=1)

# cadrul care incapsuleaza intregul continut
content = ttk.Frame(root)

# caseta text care afiseaza raspunsurile la intrebari
response_widget = Text(content, height=10, width=50)

# eticheta text din partea dreapta
question_label = ttk.Label(content, text="Profesorul intreaba:")

# caseta de introducere text cu care se preia intrebarea de la utilizator
question = ttk.Entry(content, width=50)

# butoanele din dreapta-jos
ask = ttk.Button(content, text="Intreaba", command=ask_question) # la
apasare, se apeleaza functia ask_question
exitbtn = ttk.Button(content, text="Iesi", command=root.destroy) # la
apasare, se iese din aplicatie

# plasarea elementelor in layout-ul de tip grid
content.grid(column=0, row=0)
response_widget.grid(column=0, row=0, columnspan=3, rowspan=4)
question_label.grid(column=3, row=0, columnspan=2)
question.grid(column=3, row=1, columnspan=2)
ask.grid(column=3, row=3)
exitbtn.grid(column=4, row=3)

# bucla principala a interfetei grafice care asteapta evenimente de la
utilizator
root.mainloop()

```

MessageManagerMicroservice

```

package com.sd.laborator

import java.io.BufferedReader
import java.io.InputStreamReader
import java.net.ServerSocket
import java.net.Socket
import kotlin.concurrent.thread
import kotlinx.coroutines.*
import kotlinx.coroutines.sync.Mutex

```



```

import kotlinx.coroutines.sync.withLock
import java.util.*

class MessageManagerMicroservice {
    private val subscribers: HashMap<Int, Socket>
    private lateinit var messageManagerSocket: ServerSocket

    companion object Constants {
        const val MESSAGE_MANAGER_PORT = 1500
    }

    init {
        subscribers = hashMapOf()
    }

    private fun broadcastMessage(message: String, except: Int) {
        subscribers.forEach {
            it.takeIf { it.key != except }
                ?.value?.getOutputStream()?.write((message +
"\n").toByteArray())
        }
    }

    private fun respondTo(destination: Int, message: String) {
        subscribers[destination]?.getOutputStream()?.write((message +
"\n").toByteArray())
    }

    fun run() = runBlocking {
        // se porneste un socket server TCP pe portul 1500 care asculta pentru
conexiuni
        messageManagerSocket = ServerSocket(MESSAGE_MANAGER_PORT)

        println("Se asteapta conexiuni si mesaje...")
        launch(newSingleThreadContext("Message Manager for Teacher")) {
            messageThreadEntryPoint(messageManagerSocket)
        }
    }

    private fun messageThreadEntryPoint(socket: ServerSocket) = runBlocking {
        while (true) {
            val clientConnection = socket.accept()
            // Using a launch 'ceuse why not?
            launch(Dispatchers.Default) {
                println("Subscriber conectat:
${clientConnection.inetAddress.hostAddress}:${clientConnection.port}")

                // save it

```

```

        subscribers[clientConnection.port] = clientConnection

        val bufferReader =
BufferedReader(InputStreamReader(clientConnection.inputStream))

        while (true) {
            val receivedMessage = bufferReader.readLine()

            if (receivedMessage == null) {
                println("Subscriber-ul ${clientConnection.port} a fost
deconectat.")

                synchronized(subscribers) {
                    subscribers.remove(clientConnection.port)
                }
                bufferReader.close()
                clientConnection.close()
                break
            }

            println("[THREAD ${Thread.currentThread().name}] -> Primit
mesaj: $receivedMessage")
            if(receivedMessage.startsWith("ping")) {

clientConnection.getOutputStream().write("pong\n".toByteArray())
                continue
            }
            val (messageType, messageDestination, messageBody) =
receivedMessage.split(" ", limit = 3)

            when (messageType) {
                "intrebare" -> {
                    println("Trimit broadcast")
                    broadcastMessage("intrebare ${clientConnection.port}
$messageBody", except = clientConnection.port)
                }
                "raspuns" -> {
                    respondTo(messageDestination.toInt(), messageBody)
                }
            }
        }
    }
}

fun main(args: Array<String>) {
    val messageManagerMicroservice = MessageManagerMicroservice()
    messageManagerMicroservice.run()
}

```

